Oficina Mecânica · Domain-Driven Design

FIAP SOAT · Tech Challenge Fase 1 · Event Storming e diagramas levantados a partir do código entregue (repositório fiap-oficina-api)

Legenda

Legenda

Cada cor é um tipo de post-it. Em cada fluxo, a linha do tempo corre da esquerda para a direita, e cada coluna é a "receita" de um passo, de cima para baixo: ator, read model, comando, agregado, evento e política.

Post-it	Significado
Evento de domínio	Algo que aconteceu. Verbo no particípio: "OS Aberta"
Fato registrado	Fato gravado pelo sistema, mas não publicado como evento no código
Comando	Intenção de um ator. Imperativo: "Enviar Orçamento"
Agregado	Recebe o comando e garante as regras: "OrdemServico"
Política	Reação automática: "quando X, então Y"
Read model	Informação consultada para decidir
Ator	Quem executa o comando
Sistema externo	Fora do domínio: notificador do cliente
Hotspot	Regra polêmica, risco ou decisão registrada

Fluxo 1 · Criação da OS

Fluxo 1 · Criação da Ordem de Serviço

Do cliente chegando à oficina até a decisão sobre o orçamento. Rotas: POST /admin/clientes, POST /admin/ordens-servico, /servicos, /pecas, /orcamento/enviar, /public/ordens-servico/{codigo}/orcamento/aprovar|rejeitar

1 · Identificar cliente	2 · Abrir OS	3 · Diagnóstico	4 · Orçamento	5 · Decisão
Atendente	Atendente	Mecânico	Mecânico / Atendente	Cliente (API pública) ou Atendente
Cliente por CPF/CNPJ	Abrir OS (CPF/CNPJ, placa, serviços, peças)	Catálogo de serviços · Peças e insumos (preço vigente)	Enviar Orçamento	Acompanhamento da OS
Cadastrar Cliente	Quando a placa é nova, então cadastrar veículo	Incluir Serviço / Incluir Peça	OrdemServico (gera Orcamento)	Aprovar Orçamento
Cliente	Veiculo	OrdemServico	Orçamento Enviado · status Aguardando aprovação	Rejeitar Orçamento
Cliente Cadastrado (fato registrado)	Veículo Cadastrado (fato registrado)	Item Incluído na OS (fato registrado)	Quando orçamento enviado, então notificar cliente	OrdemServico
Cliente precisa existir antes da OS: abrir OS identifica o cliente, mas não o cadastra	OrdemServico	Quando o 1º item é incluído, então iniciar diagnóstico	Notificador do cliente (log no MVP)	Orçamento Aprovado → Execução Iniciada
	OS Aberta · status Recebida	Diagnóstico Iniciado · status Em diagnóstico	Orçamento é calculado pelo sistema e exige ao menos 1 item	Orçamento Rejeitado → OS encerrada sem execução
	Placa já vinculada a outro cliente → recusa (409)	Preço congelado na inclusão: reajuste do catálogo não altera a OS		Aprovação baixa o estoque na mesma transação (ver Fluxo 3)
		Serviço inativo não pode ser incluído		

Fluxo 2 · Acompanhamento da OS

Fluxo 2 · Acompanhamento da Ordem de Serviço

Execução, acompanhamento pelo cliente, entrega e métricas. Rotas: GET /public/ordens-servico/{codigo}, POST /admin/ordens-servico/{id}/finalizar e /entregar, GET /admin/ordens-servico, GET /admin/metricas/tempo-medio-execucao

1 · Acompanhar	2 · Execução	3 · Finalização	4 · Entrega	5 · Gestão	Toda transição
Cliente	Orçamento Aprovado (vem do Fluxo 1)	Mecânico	Atendente	Administrador	Status Alterado (ordem-servico.status-alterado)
Acompanhamento da OS (status, orçamento, histórico, documento mascarado)	Execução Iniciada · status Em execução	Finalizar OS	Entregar Veículo	Lista e detalhe de OS (filtro por status e cliente)	Quando o status muda, então avisar o cliente
Cliente só vê a própria OS: código + CPF/CNPJ; divergência → 404	Tempo de execução começa na aprovação (inicioExecucao)	OrdemServico	OrdemServico	Tempo médio de execução (geral e por serviço)	Notificador do cliente (log no MVP)
		Serviços Finalizados · status Finalizada	Veículo Entregue · status Entregue	Tempo de execução = aprovação → finalização	Eventos publicados só após o commit; falha ao notificar não desfaz a operação
		Transição fora da ordem é recusada pelo agregado (422)			

Fluxo 3 · Peças e insumos

Fluxo 3 · Gestão de peças e insumos

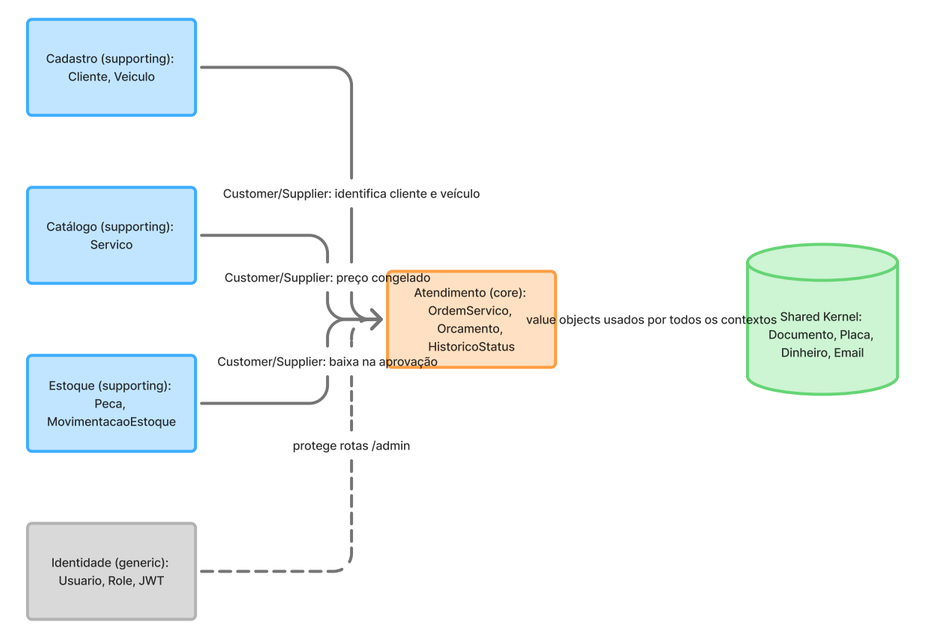
Cadastro, reposição, uso na OS e baixa de estoque. O contexto de Estoque não publica eventos de domínio no MVP: seus fatos ficam registrados como MovimentacaoEstoque (trilha de auditoria imutável). Rotas: POST /admin/pecas, POST /admin/pecas/{id}/entradas, GET /admin/pecas/{id}/movimentacoes, GET /admin/pecas?abaixoDoMinimo=true

1 · Cadastro	2 · Reposição	3 · Uso na OS	4 · Baixa	5 · Auditoria
Administrador	Administrador	Mecânico	Orçamento Aprovado (vem do Fluxo 1)	Administrador
Cadastrar Peça/Insumo (código, tipo, preço, saldo, mínimo)	Peças abaixo do mínimo	Peças e insumos (saldo, preço)	Quando orçamento aprovado, então baixar peças da OS (síncrono, mesma transação)	Movimentações da peça (entradas e saídas)
Peca	Registrar Entrada	Incluir Peça na OS	Peca	
Peça Cadastrada (fato registrado)	Peca	OrdemServico	Estoque Baixado (movimentação SAÍDA) (fato registrado)	
Código único; tipo PECA ou INSUMO; saldo e mínimo ≥ 0	Entrada de Estoque Registrada (movimentação ENTRADA) (fato registrado)	Item Incluído na OS (fato registrado)	Estoque Insuficiente → aprovação desfeita (rollback) (fato registrado)	
	Sem alerta automático de reposição: o MVP só sinaliza abaixoDoMinimo	Incluir peça NÃO reserva estoque: o saldo só é verificado na aprovação	Concorrência: UPDATE atômico + CHECK saldo ≥ 0 no banco	

Diagramas

Mapa de contextos (bounded contexts)

Atendimento é o core domain e consome os contextos de suporte por chamada em processo (monólito). Shared Kernel: value objects comuns.



Cadastro (supporting): Cliente, Veiculo

Catálogo (supporting): Servico

Estoque (supporting): Peca, MovimentacaoEstoque

Identidade (generic): Usuario, Role, JWT

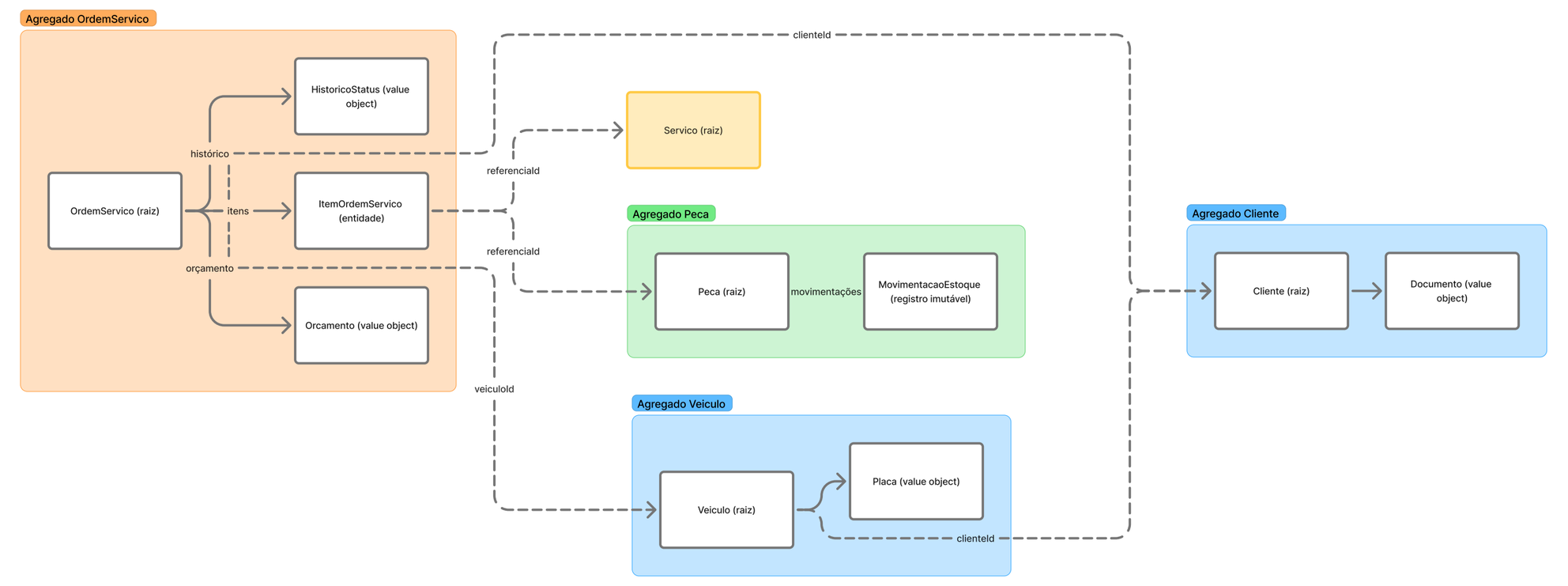
Atendimento (core): OrdemServico, Orcamento, HistoricoStatus

Shared Kernel: Documento, Placa, Dinheiro, Email

Customer/Supplier: identifica cliente e veículo
Customer/Supplier: preço congelado
Customer/Supplier: baixa na aprovação
protege rotas /admin
value objects usados por todos os contextos

Agregados, entidades e objetos de valor

Seta cheia: composição dentro do agregado. Seta tracejada: referência por identidade a outro agregado (só o id).



Agregado OrdemServico

OrdemServico (raiz)

HistoricoStatus (value object)

ItemOrdemServico (entidade)

Orcamento (value object)

Servico (raiz)

Agregado Peca

Peca (raiz)

MovimentacaoEstoque (registro imutável)

Agregado Veiculo

Veiculo (raiz)

Placa (value object)

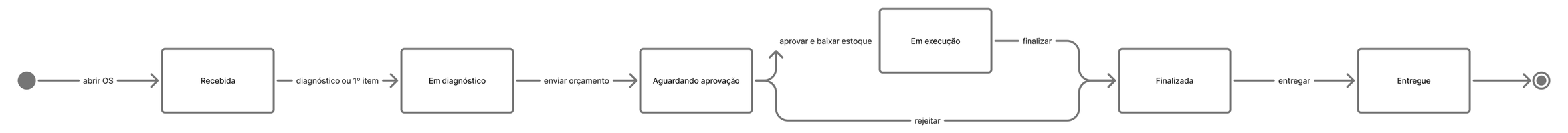
Agregado Cliente

Cliente (raiz)

Documento (value object)

Máquina de estados da Ordem de Serviço

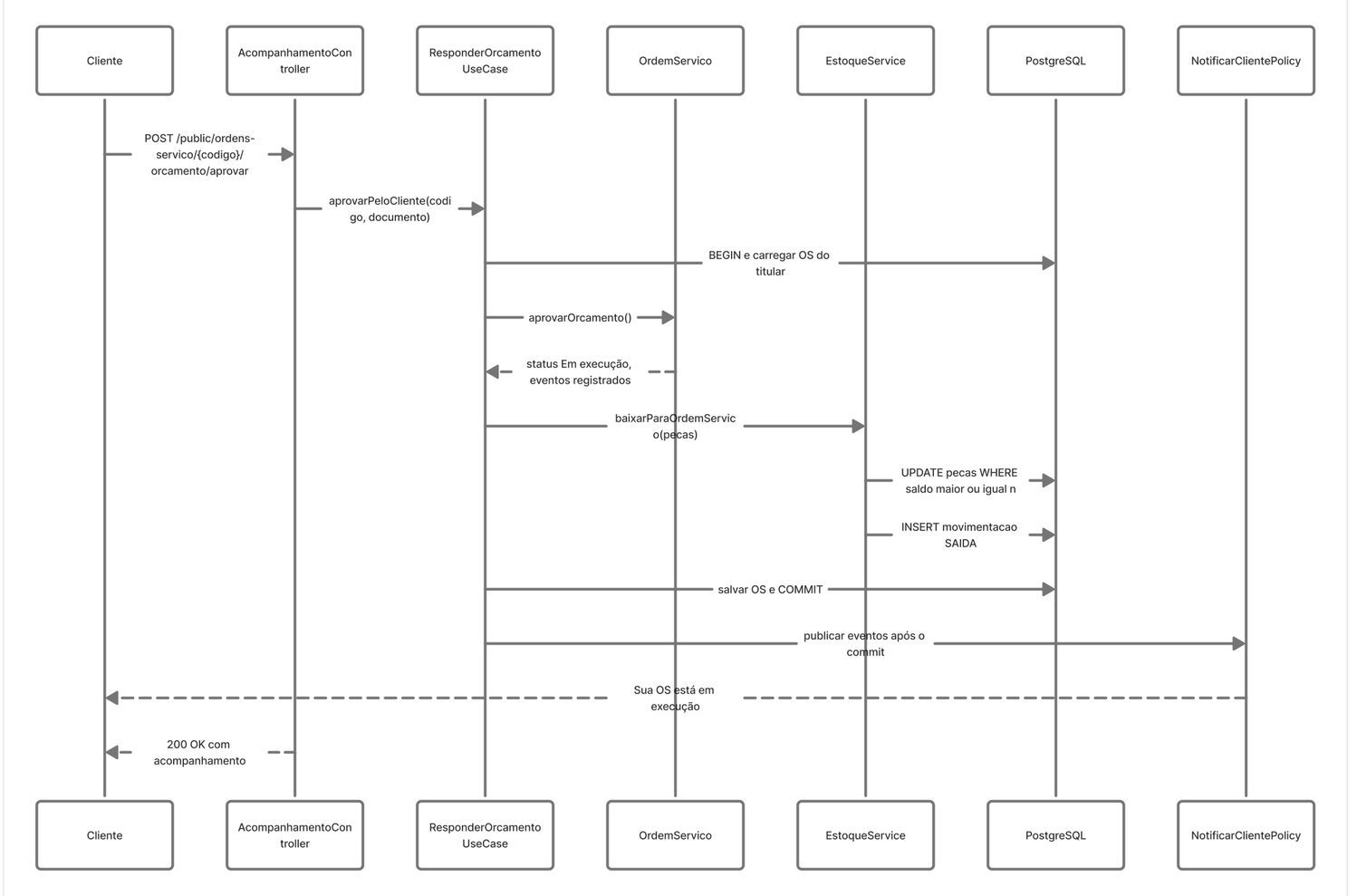
Transições aplicadas pelo agregado (TRANSICOES_PERMITIDAS); qualquer outra é recusada com HTTP 422. Itens só são editáveis em Recebida e Em diagnóstico.



Sequência: aprovação do orçamento

Status e baixa de estoque na MESMA transação; eventos publicados só depois do commit.

Sequência: aprovação do orçamento



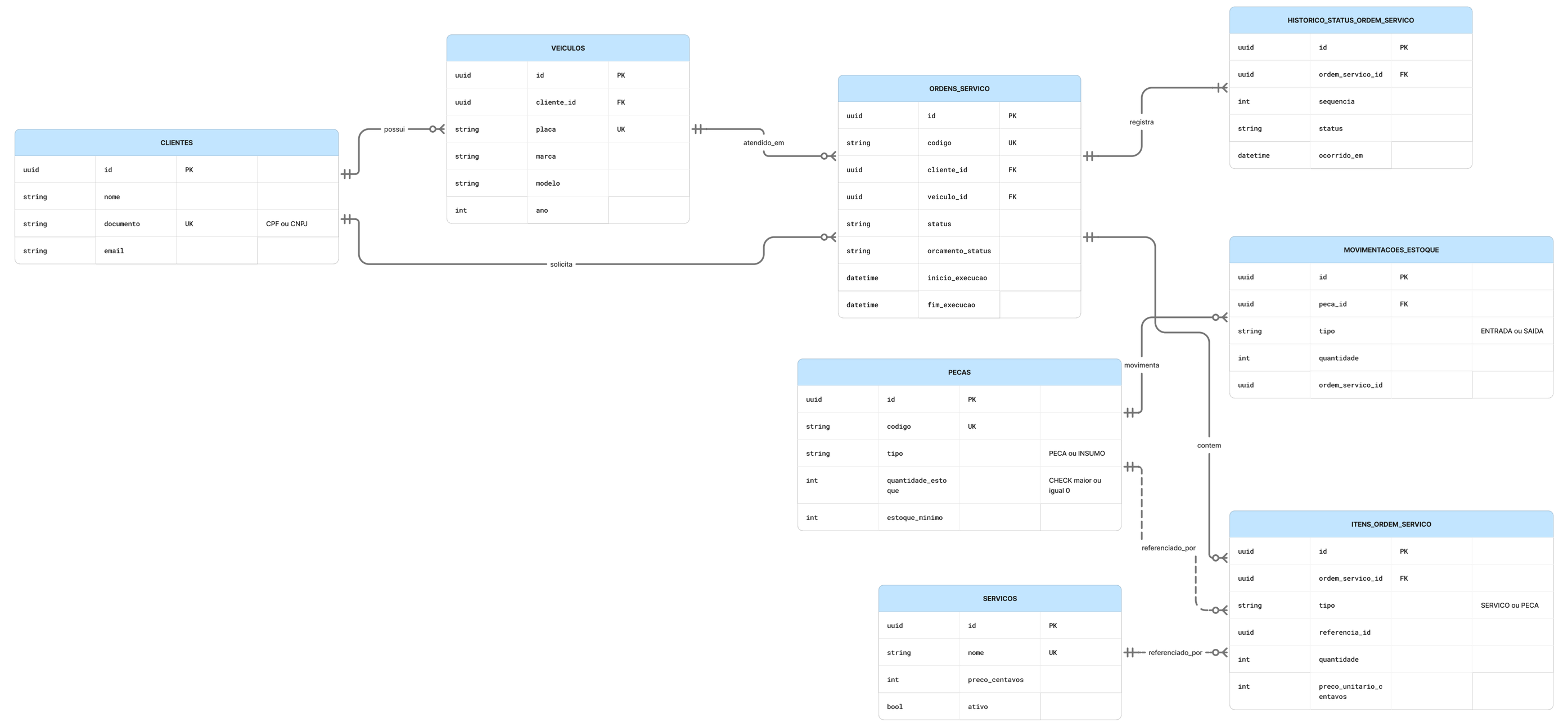
Cliente · AcompanhamentoController · ResponderOrcamentoUseCase · OrdemServico · EstoqueService · PostgreSQL · NotificarClientePolicy

- POST /public/ordens-servico/{codigo}/orcamento/aprovar
- aprovarPeloCliente(codigo, documento)
- BEGIN e carregar OS do titular
- aprovarOrcamento()
- status Em execução, eventos registrados
- baixarParaOrdemServico(pecas)
- UPDATE pecas WHERE saldo maior ou igual n
- INSERT movimentacao SAIDA
- salvar OS e COMMIT
- publicar eventos após o commit
- Sua OS está em execução
- 200 OK com acompanhamento

Caminho de erro: se faltar saldo em qualquer peça, EstoqueInsuficienteError → ROLLBACK. A OS continua Aguardando aprovação, nada é baixado e a API responde 422. Comprovado no teste e2e de rollback.

Modelo de dados (PostgreSQL)

Atributos principais. Esquema completo em prisma/schema.prisma e prisma/migrations.



CLIENTES

uuid	id	PK	
string	nome		
string	documento	UK	CPF ou CNPJ
string	email		

VEICULOS

uuid	id	PK
uuid	cliente_id	FK
string	placa	UK
string	marca	
string	modelo	
int	ano	

ORDENS_SERVICO

uuid	id	PK
string	codigo	UK
uuid	cliente_id	FK
uuid	veiculo_id	FK
string	status	
string	orcamento_status	
datetime	inicio_execucao	
datetime	fim_execucao	

HISTORICO_STATUS_ORDEM_SERVICO

uuid	id	PK
uuid	ordem_servico_id	FK
int	sequencia	
string	status	
datetime	ocorrido_em	

MOVIMENTACOES_ESTOQUE

uuid	id	PK	
uuid	peca_id	FK	
string	tipo		ENTRADA ou SAIDA
int	quantidade		
uuid	ordem_servico_id		

PECAS

uuid	id	PK	
string	codigo	UK	
string	tipo		PECA ou INSUMO
int	quantidade_estoque		CHECK maior ou igual 0
int	estoque_minimo		

SERVICOS

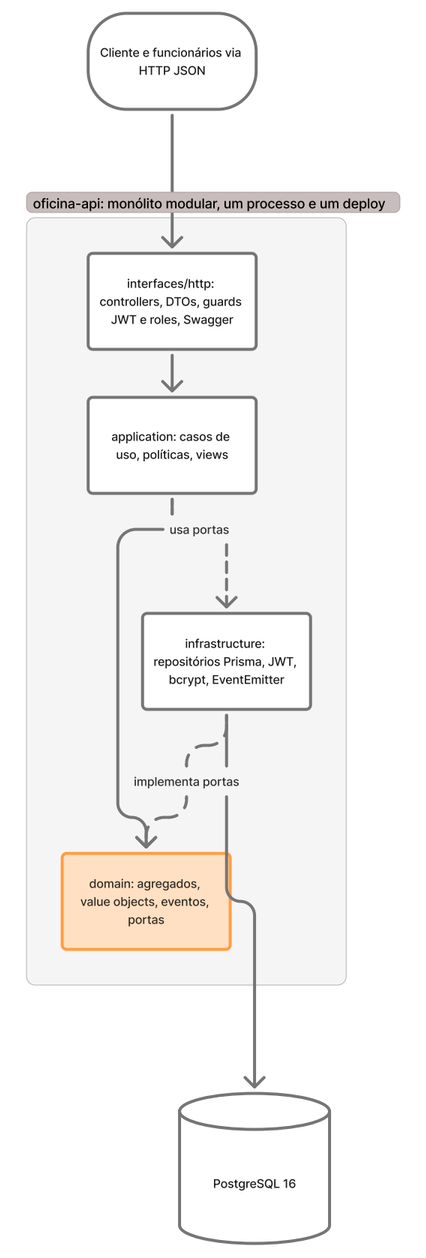
uuid	id	PK
string	nome	UK
int	preco_centavos	
bool	ativo	

ITENS_ORDEM_SERVICO

uuid	id	PK	
uuid	ordem_servico_id	FK	
string	tipo		SERVICO ou PECA
uuid	referencia_id		
int	quantidade		
int	preco_unitario_centavos		

Arquitetura em camadas (monólito modular)

Cada bounded context é um módulo NestJS com as mesmas 4 camadas; as dependências apontam para o domínio.



Cliente e funcionários via HTTP JSON

oficina-api: monólito modular, um processo e um deploy

interfaces/http: controllers, DTOs, guards JWT e roles, Swagger

application: casos de uso, políticas, views

infrastructure: repositórios Prisma, JWT, bcrypt, EventEmitter

domain: agregados, value objects, eventos, portas

PostgreSQL 16